AUTOS COLOMBIA

Gestión de la entrada y salida de vehículos

Documento de entrega

Institución Universitaria Digital de Antioquia

Equipo: Henao Juan David · Uribe Jorge Eliecer · Valencia Santiago Andres

2026

https://github.com/andresvd95/IngSoftIUDig

Introducción

Este documento presenta los entregables de la primera iteración de Autos Colombia, un sistema de información para apoyar la operación diaria de un parqueadero por mensualidad. El alcance se concentra en registrar la entrada y salida de vehículos, asignar y liberar celdas, consultar estancias activas y documentar novedades.

La solución combina un backend Spring Boot con persistencia MySQL y migraciones Flyway, y un frontend Angular responsive. La interfaz puede demostrarse con datos simulados mientras se completa la integración del contrato de API. El código fuente, diagramas de datos, mockups y documentos de diseño se mantienen en el repositorio GitHub.

Objetivo del documento

Consolidar en un solo PDF la evidencia disponible, el alcance funcional y el estado de cumplimiento de los nueve entregables solicitados.

Entregables de la iteración 1

La siguiente matriz relaciona cada requisito con su evidencia dentro del repositorio.

No.	Entregable	Evidencia / alcance	Estado
1	Requerimientos funcionales y no funcionales	PRD-FRONTEND.md: alcance, flujos, estados, responsive y accesibilidad.	Incluido
2	Historias de usuario	Historias y criterios de aceptación asociados a entrada, salida, estancias, celdas y novedades.	Incluido
3	Arquitectura del sistema	architecture.png y TDD-FRONTEND.md: Angular, API Spring Boot, MySQL, Flyway y Docker.	Incluido
4	Mockups en Figma	docs/mockups/: pantallas de login, resumen, entradas, salidas, celdas, novedades y detalle.	Incluido
5	MER, MR y modelo físico	autos-colombia-diagrama-MER.png, autos-colombia-diagrama-ER.png, autos-colombia-diagrama-fisico.png y SQL inicial.	Incluido
6	Diagrama de casos de uso StarUML	Relaciones y casos descritos en el alcance; falta anexar archivo editable .mdj.	Pendiente .mdj
7	Diagrama de clases StarUML	Dominios, entidades y servicios implementados en autos-colombia/src/main/java.	Pendiente .mdj
8	Diagrama de paquetes StarUML	Paquetes reflejados en la organización del código fuente Java y TDD.	Pendiente .mdj
9	Código ejecutable y repositorio GitHub	Backend Spring Boot, frontend Angular, pruebas y README en el repositorio público.	Incluido

Requerimientos y necesidades del usuario

Requerimientos funcionales

Código	Requisito
RF-01	Registrar el ingreso de un vehículo, asociando placa, cliente, operador y celda disponible.
RF-02	Registrar la salida de un vehículo y liberar la celda asignada.
RF-03	Consultar vehículos dentro, estancias activas y detalle de una estancia.
RF-04	Consultar el estado de las celdas: disponible, ocupada o fuera de servicio.
RF-05	Registrar y consultar novedades asociadas a una estancia.
RF-06	Validar reglas de negocio: vehículo ya dentro, celda no disponible y sesión activa inexistente.

Requerimientos no funcionales

Área	Criterio
Usabilidad	Flujos principales accesibles desde móvil, tablet y escritorio; mensajes de error recuperables.
Accesibilidad	HTML semántico, foco visible, teclado, etiquetas y contraste WCAG AA.
Rendimiento	Consultas paginadas y respuesta adecuada para la operación de un parqueadero.
Mantenibilidad	Separación por dominio, servicios, mapeadores y configuración externa de la API.
Despliegue	Frontend compilable y servible con Nginx/Docker; backend ejecutable con Maven Wrapper.
Trazabilidad	Migraciones versionadas con Flyway y documentación de ejecución en README.

Historias de usuario

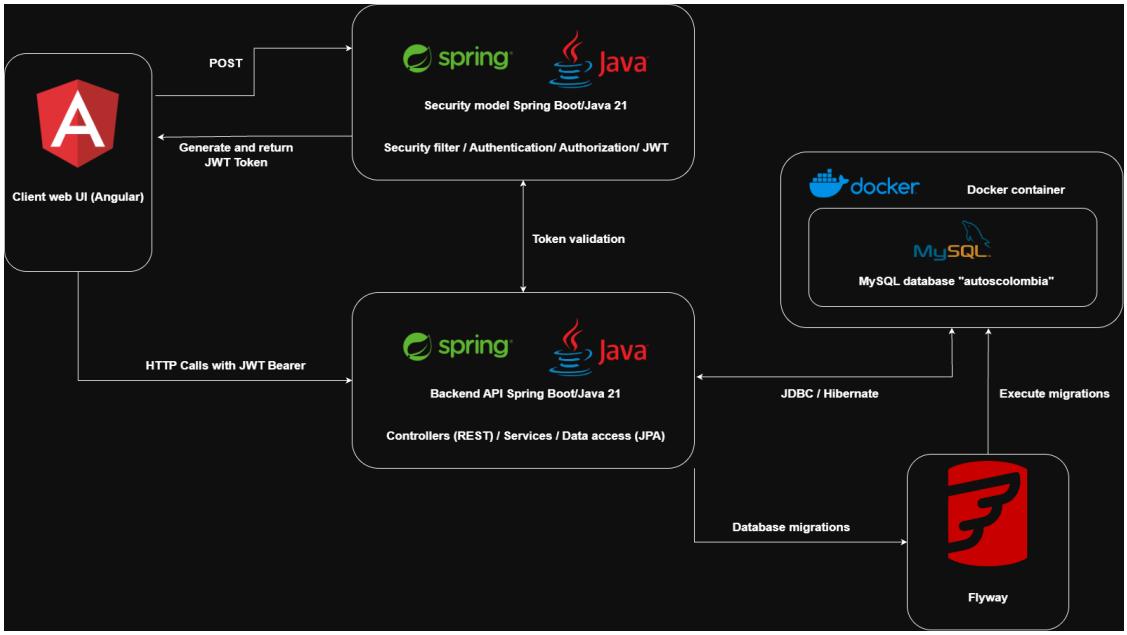
ID	Como	Quiero	Para
HU-01	Operador	Registrar la entrada de un vehículo y asignar una celda.	Controlar la ocupación desde el primer momento.
HU-02	Operador	Consultar vehículos dentro y filtrar por placa.	Encontrar rápidamente una estancia activa.
HU-03	Operador	Registrar la salida de un vehículo.	Liberar la celda y mantener trazabilidad.
HU-04	Operador	Consultar el estado de las celdas.	Elegir una celda compatible y disponible.
HU-05	Operador	Registrar una novedad en una estancia.	Dejar constancia de lo ocurrido.
HU-06	Administrador	Consultar movimientos y datos relacionados.	Supervisar la operación diaria.

Criterios comunes de aceptación

Cada flujo debe validar datos obligatorios, mostrar estados de carga, vacío, error y éxito, conservar el contexto ante errores recuperables y evitar operaciones duplicadas. Una entrada exitosa actualiza las estancias y las celdas; una salida elimina la estancia activa y libera visualmente la celda.

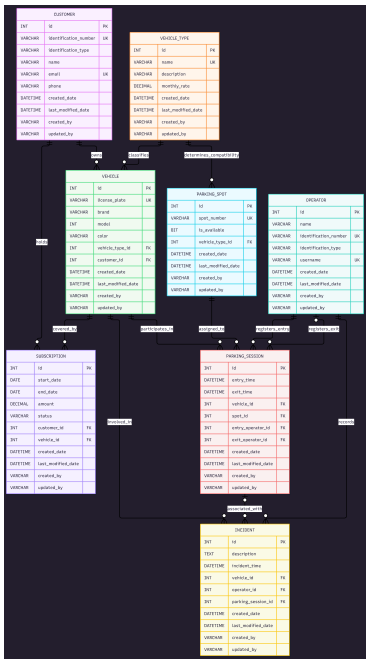
Arquitectura y modelos de datos

La arquitectura separa el cliente web, la API y la persistencia. El backend expone servicios REST y organiza el dominio en vehículos, tipos de vehículo, celdas y sesiones de parqueo. Flyway versiona el esquema MySQL.

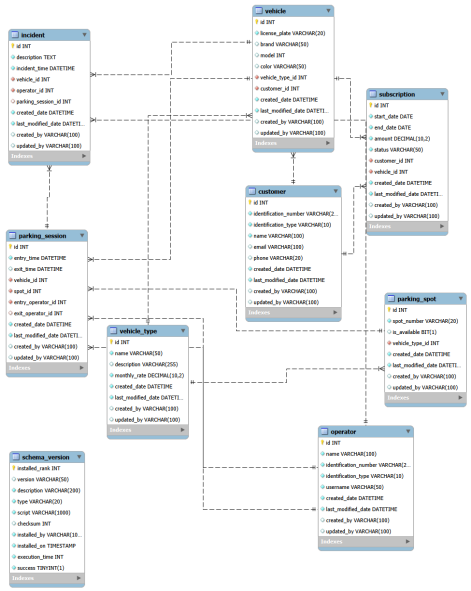


Arquitectura propuesta

Modelo entidad relación y modelo físico



Modelo entidad relación



Modelo físico

Diseño StarUML y código fuente

El diseño requerido para StarUML se organiza en tres vistas: casos de uso (operador y administrador frente a entradas, salidas, celdas y novedades), clases (entidades, servicios, repositorios y DTO) y paquetes (capas de controller, service, persistence y subdominios). La implementación Java refleja estas responsabilidades en su estructura de paquetes.

Componente	Implementación comprobable
Backend	Spring Boot 4.1.1, Java 21, Spring Data JPA, Flyway y MySQL.
Endpoints	REST para tipos, celdas y sesiones; alta y baja de vehículos.
Frontend	Angular standalone, SCSS, rutas operativas y datos simulados para demostración.
Pruebas	mvnw test con H2 en memoria para que el contexto de Spring sea reproducible sin Docker.

Ejecución

Backend (con MySQL y variables de entorno): .\mvnw.cmd spring-boot:run. Frontend: npm install y npm start. Pruebas: .\mvnw.cmd test. La documentación completa se encuentra en README.md, PRD-FRONTEND.md y TDD-FRONTEND.md.

Conclusiones

- La Iteración 1 define un flujo coherente para registrar entradas y salidas, consultar ocupación y asociar novedades a una estancia.

- El repositorio contiene una base ejecutable con frontend Angular, backend Spring Boot, persistencia MySQL, migraciones Flyway, pruebas y documentación técnica.

- Los requisitos, historias, arquitectura, mockups y modelos de datos están documentados y cuentan con archivos de soporte en el repositorio.

- Para declarar el paquete StarUML completamente cerrado todavía deben agregarse los archivos editables .mdj de casos de uso, clases y paquetes; este PDF deja esa brecha identificada para evitar confundir documentación textual con el artefacto editable.

Repositorio y evidencia

Código fuente y documentación de la Iteración 1:

https://github.com/andresvd95/IngSoftIUDig

Este enlace está incorporado como vínculo clicable en el PDF y permite verificar la versión publicada del proyecto.